Name of your Institute: Manipal Institute of Technology, Manipal

Challenge Area: Track 6 – The Next Billion Builders

Your Team Name: br0c0de

Registered Email ID of your Team Lead: disha1.mitmpl2024@learner.manipal.edu

PROJECT TITLE: AGENTIC NEXUS

The AI-Powered Multi-Agent Application Builder for the Next Billion Builders

THE TEAM

Our team came together through a shared passion for AI and problem-solving. We've worked together before and even won a hackathon in our college as a team, which helped us realise how well our skills complement each other. That experience inspired us to reunite for AI Unlocked, driven by the same goal of building meaningful technology and empowering others to turn ideas into reality.

THE CONCEPT

The world is full of people with great ideas and no way to build them, and these are precisely the next billion builders the world needs. A healthcare researcher who spots a workflow problem, a student with a startup concept, an entrepreneur who knows exactly what their customers need- none of them can act on those ideas without either learning five different technical disciplines or hiring a team to do it for them. This is a massive, largely invisible loss: countless valuable applications that never get built simply because the barrier to entry is too high. As AI makes software more powerful than ever, it's also increasingly important that not just engineers, but also the people who understand real-world problems, can participate in building solutions.

Agentic Nexus solves this by letting anyone, even those with no technical expertise, describe what they want in plain language and walk away with a fully tested, production-ready application deployed on Azure. Built on Microsoft's AI platform, it orchestrates specialised AI agents coordinated by a central Director AI that keeps the user informed and in control throughout. Features like a Self-Healing Debugger, Cost Optimiser, Digital Twin simulation, and a Data-to-App Pipeline ensure the experience is fast, safe, and accessible to all. The goal is simple: give everyone the power to build.

TARGET AUDIENCE OR MARKET

Agentic Nexus targets four primary segments:

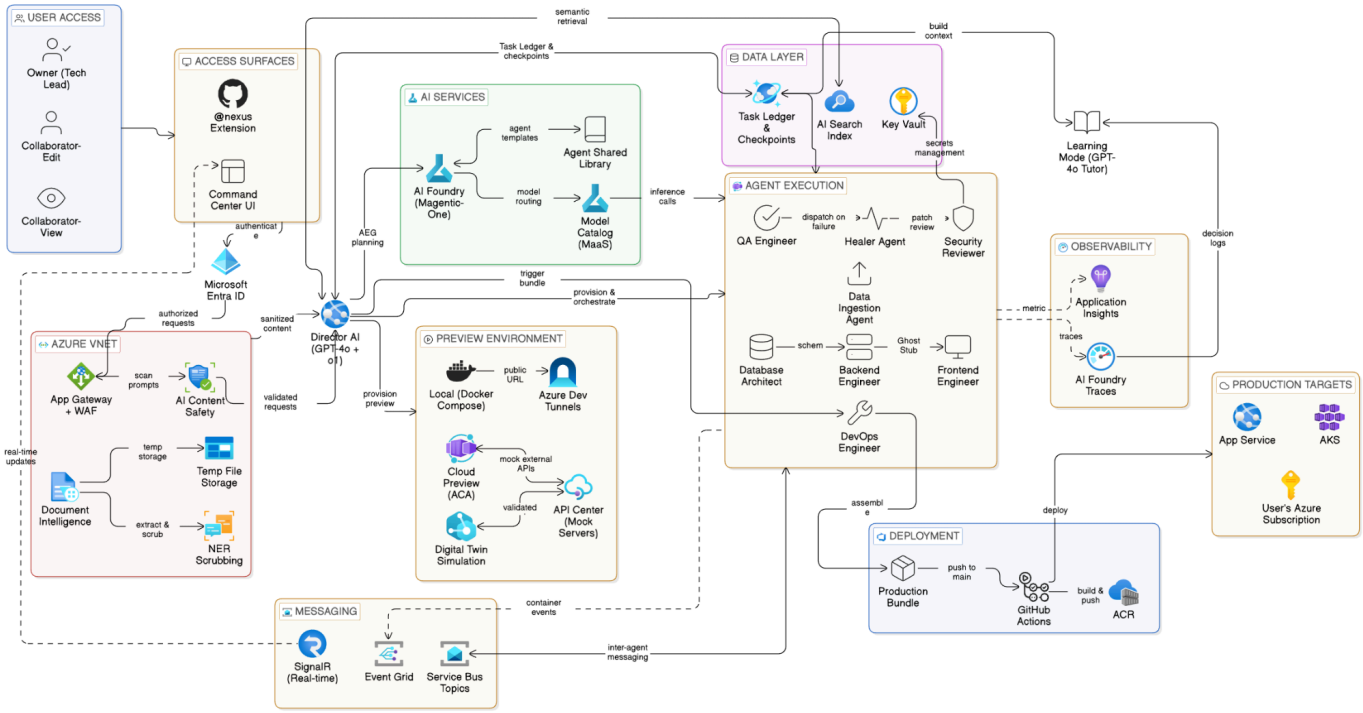
- **Non-developer domain experts:** Solo entrepreneurs, product managers, and researchers, i.e. those with deep domain knowledge but zero coding ability. Approximately 12% of working-age adults in India are actively trying to start a business (GEM Consortium).
- **Students & Employees:** 33M+ developers worldwide, plus 43.3M students in Indian higher education, including 212,000 PhD researchers (PIB 2022). These are all defined by one shared constraint: lack of team capacity. Nexus replaces an entire engineering team for prototype and MVP-stage builds.
- **Small Businesses:** India hosts 63M MSME enterprises (57M registered on Udyam/UAP) with real operational needs (booking tools, inventory, portals) but no dev budget. Nexus essentially gives them a technical co-founder on demand.
- **Enterprise engineering teams:** IDC research shows developers dedicate only about 16% of their time to actual development, and the rest is coordination overhead. With 140,803 DPIIT-recognised startups in India alone (June 2024), this segment is large and fast-growing.

Geography: Global, with initial emphasis on North America and Europe (highest Azure penetration). Market: low-code/no-code + AI developer tools projected at $65B by 2027 (Gartner 2024), AI-assisted development is the fastest-growing sub-segment.

PERSONAS

Priya, 28 (Solo Founder) *Building a jewellery inventory app, no engineering team* **Pain:** Was quoted ₹5-8L by dev agencies. Given that the average developer salary ₹8-10L/yr, hiring is off the table entirely. **Value with Nexus:** Explains requirements in simple terms, watches agents build out module by module, and provides feedback via conversation. Builds a full-stack, cloud-deployed app in hours through @nexus in VS Code, no team needed.	**Kwame, 22 (Final Year Business Student)** *Creating a farmer-to-buyer marketplace, no technical background* **Pain:** A student in a rural area with no mentors in the area. No-code platforms don't support custom SMS integrations or local payment gateways. Idea stuck in a notebook. **Value with Nexus:** Orchestrator helps break down the platform into modules like marketplace, SMS, payments, and farmer dashboard. Kwame can approve steps without looking into any code. Deploys a production-ready marketplace with specialised agents working in parallel.

HOW IT WORKS



Agentic Nexus is a hierarchical multi-agent orchestration platform. A central Director AI (GPT-4o + o1-preview) owns the master plan and all user communication, but never writes code directly. It provisions up to 8 specialised agents from an Azure AI Foundry Agent Catalogue, each running in an isolated Azure Container Apps container, and supervises their parallel execution across a 7-stage pipeline.

Stage 1 - Requirements & Guardrail Dialogue: The Director opens a multi-turn clarification loop with the user, building a Task Ledger: a versioned JSON document stored in Azure Cosmos DB that serves as the single source of truth for the entire project. It identifies ambiguities across five axes: functional scope, technical constraints, target users, integration requirements, and quality/compliance criteria. Before planning begins, a Guardrail Dialogue engine proactively surfaces risk signals such as architectural mismatches, security vulnerabilities, cost explosion risks, dependency conflicts, and platform mismatches as structured, non-blocking challenges. The user retains full authority to override any guardrail, but every override is logged in the Task Ledger's revision history for later auditability. A Validator Agent then performs a Feasibility Check against a set of architectural anti-patterns before the project proceeds.

Stage 2 - Agent Execution Graph (AEG) Planning: The Director decomposes the Task Ledger into modules (e.g., Auth Service, REST API, Database Schema, Frontend, CI/CD Pipeline) and generates an Agent Execution Graph. The Agent Execution Graph is a directed acyclic graph where each node is a specialised agent, and each edge encodes a data dependency. For each node, the Director specifies the agent role, input requirements, output contracts, token budget, and model preference. Dependency edges are classified as Blocking (downstream cannot start without real output, e.g., DB migrations before the API can run) or Ghost-eligible (downstream can begin with a provisional stub). The AEG is rendered as a visual dependency diagram in the Command Centre UI. A mandatory Human-in-the-Loop checkpoint requires explicit user approval of the AEG before any cloud resource is provisioned. Structural changes after approval require a Delta AEG re-approval flow.

Stage 3 - Agent Provisioning & Token-Thrifty Routing: Each AEG node is matched to a template in the Azure AI Foundry Agent Catalogue using DiskANN semantic search over vector embeddings. If no suitable template exists, the Director constructs one with a custom system prompt and MCP tool manifest, seeding it into the catalogue for future reuse. The Token-Thrifty Optimiser then assigns a model tier to every task before dispatch: simple tasks (CSS, boilerplate, documentation) are assigned to Microsoft Phi-4; intermediate tasks (CRUD logic, unit tests) are given to GPT-4o; complex tasks (auth architecture, security review) are handled by GPT-5 or Claude Sonnet 4.5; high-reasoning tasks (planning, root cause analysis) are reserved for heavy models like Claude Opus. If a task assigned to a smaller model fails self-validation, a Model Escalation Policy automatically promotes it one tier at a time, up to a point, in order to prevent a hallucination loop where a weak model attempts to fix its own broken output. MCP tool connections are scope-validated before any agent is granted permissions.

Stage 4 - Parallel Execution, Ghost Handshakes & Self-Healing: Up to 8 agents run concurrently inside Azure Container Apps. The Ghost Handshake Protocol eliminates the classic parallel-dependency stall: the moment an upstream agent starts, it publishes a provisional schema-conformant stub of its output, unblocking downstream agents immediately. When real output is ready, downstream agents receive a GHOST_RESOLVED event over their Azure Service Bus A2A channel and seamlessly swap the stub for the real contract, cutting total build time. If a real output diverges significantly from its stub (a Ghost Divergence), the Director evaluates whether a targeted rewrite or fresh generation is cheaper and explains the choice in the live AEG view. Agents coordinate over named Service Bus queues; additive contract changes broadcast DELTA_UPDATE messages to all subscribers. Every completed task writes atomically to both Cosmos DB (checkpoint) and Git (commit), enabling Two-Layer Rollback from any point in the Build Timeline. The Self-Healing Debugger dispatches a Healer Agent on any test failure: it performs root-cause analysis using full Foundry Observability traces, generates a minimal patch, and routes it through the Security

Reviewer for OWASP validation before re-running the affected tests. If a Healer patch causes a secondary failure (a Healer Cascade), the Director takes ownership of the compound problem rather than spawning a new independent Healer chain. The global Healer activation count is capped per build session to prevent runaway repair loops.

Stage 5 - Monitoring, Persistence & Agent Rating: The Command Centre Dashboard renders a live AEG view with each node colour-coded by state (PENDING / RUNNING / BLOCKED / COMPLETED / FAILED), a live token/cost counter, and a streaming preview for frontend modules. Rather than dissolving agents on task completion, the Director checkpoints their full state, including conversation history, artefacts, tool call logs, and performance metrics to Cosmos DB. This enables Rehydration: if a bug is discovered after an agent has been dissolved, it can be resurrected with full memory intact, patched, and its downstream validation re-run without replanning. After each engagement, the Director computes a multi-dimensional Reputation Score (task success rate, token efficiency, contract fidelity, downstream satisfaction, time-to-completion) and writes it back to the catalogue entry, so better-performing agents are preferentially selected for future builds.

Stage 6 - Live Preview & Iterative Feedback: Before any production infrastructure is provisioned, the user interacts with the actual generated application. On capable hardware, the Deployment Agent spins up a Local Preview via Docker Compose + Azure Dev Tunnels, the real app running against a seeded local database, not a mock. On constrained hardware (below 8 GB RAM, or mobile), an equivalent Cloud Preview automatically provisions an identical stack as an ephemeral Azure Container Apps Environment (hard 4-hour TTL, cost disclosed upfront, typically under $0.50). The user requests changes at three granularities: Cosmetic changes (colours, copy), hot-reload by rehydrating the relevant agent from its checkpoint in seconds; Behavioural changes (validation rules, new fields) trigger targeted minimal agent rehydration and re-run QA for affected modules only; Structural changes (new integrations, architectural shifts) generate a Delta AEG for re-approval before execution. Each iteration is versioned in the Task Ledger and reversible.

Stage 7 - Digital Twin Simulation & One-Click Ship: Before deployment, the user can launch a Digital Twin Simulation: the full application runs in an isolated ACA Jobs environment where every external dependency is replaced with deterministic mocks. Database calls use an in-memory SQLite instance seeded from the schema; third-party API calls are served by a local mock server. Critically, mock fidelity is guaranteed by Azure API Centre: for any API registered there, mocks are generated directly from the real OpenAPI specification rather than AI inference, ensuring schema-validated request/response contracts. For supported services (GitHub, Stripe, Azure DevOps), MCP Servers in read-only mode serve real API responses without triggering write side-effects. The simulation runs all QA test scenarios and produces a User Flow Report visualising every primary user journey. Once satisfied, a single Ship button triggers the GitHub Actions CI/CD pipeline, building the Docker image, pushing it to Azure Container Registry, and executing the Bicep-defined infrastructure deployment into the user's own Azure subscription. The user retains full ownership because Nexus operates in Zero Data Retention mode with no persistent access after the session ends.

Standout Capabilities

- **Ghost Handshake Protocol:** publishes provisional output stubs so downstream agents start immediately, cutting build time by up to 60%.
- **Token-Thrifty Model Routing:** 4-tier model selection with automatic escalation keeps API costs minimal without sacrificing quality.
- **Two-Layer Rollback:** every task checkpointed atomically in both Cosmos DB and Git; any module or full build is reversible from the Build Timeline UI.
- **Self-Healing Debugger:** automated root-cause analysis, OWASP-reviewed patch generation, and cascade protection with a per-session activation cap.
- **Digital Twin Simulation:** full integration testing with schema-validated mocks from Azure API Centre and real read-only data via MCP Servers, leading to no production side-effects.
- **Real-time Multi-User Collaboration:** Owner/Edit/View RBAC, field-level soft locks, SignalR presence indicators, and an immutable audit trail.
- **Blueprint Sharing & Agent Ecosystem:** every completed project exports a portable Blueprint to the Azure AI Foundry Shared Library; agents export as standalone Foundry Agents for community reuse.
- **Learning Mode:** GPT-4o tutor pre-loaded with the full build context, including Task Ledger, source files, agent decision log for interactive code explanation, decision archaeology, and annotated code tours.

CORE TECHNOLOGIES

Azure Service / Technology	Role in Agentic Nexus
Azure App Service	Director AI backend & Command Centre web UI
Azure Container Apps (ACA)	Up to 8 parallel agent containers; Healer & QA ephemeral jobs
Azure AI Foundry (Magentic-One)	Agent catalogue, AEG generation, multi-agent orchestration
Azure OpenAI (GPT-4o · GPT-4o-mini · o1)	Model routing via Azure AI Model Catalogue, 4-tier cost optimisation
Microsoft Phi-4	Edge-model tier for simple tasks (cheapest, lowest latency)
Azure Cosmos DB (DiskANN vector)	Task Ledger, agent checkpoints, reputation scores, and semantic agent search
Azure Service Bus + Event Grid	A2A messaging (DELTA_UPDATE, GHOST_RESOLVED, SUSPEND signals)
Azure AI Content Safety	Real-time prompt injection blocking & policy enforcement on every call
Azure Key Vault	All secrets routing; CMK for Cosmos DB; never hardcoded
Azure SignalR Service	Real-time multi-user presence, soft locks, Emergency Pause broadcast
Azure API Center	Real OpenAPI specs for schema-validated Digital Twin mock generation

GitHub Copilot Extensions API	@nexus IDE chat participant; workspace file context injection
Azure Bicep / ARM + GitHub Actions	One-click IaC deployment into the user's own subscription
Microsoft Entra ID (Azure AD)	Session ownership, RBAC roles, and immutable audit trail attribution
Azure AI Foundry Observability	Full agent traces, including model calls, tool calls, latency, and token counts
Model Context Protocol (MCP)	Standardised agent-to-tool connections; read-only simulation mode

THE BUSINESS PLAN

Four-tier gate value by projects, tokens (benchmarked to Microsoft Copilot / GitHub Copilot usage), concurrent agent count, and model quality. Crucially, every build progressively teaches users Azure: Bicep templates, Cosmos DB, ACA, Foundry Observability, etc., creating an ecosystem familiarity moat that compounds with each project and makes churning genuinely costly.

	Free	Builder	Pro	Enterprise
Projects/mo	1	5	20	Unlimited
Tokens/mo	25 K	300 K	2 M	10 M+
Concurrent agents	2	4	8	20+
Azure deployment	Guided	Guided	Full Bicep/IaC	Full + CMK/ZDR
Support	Community	Email	Priority email	Dedicated SLA

Azure retention flywheel: Free/Builder users see Azure passively (URLs, dashboards). Pro users own and edit generated Bicep + Foundry traces. Enterprise users own the full Azure stack in their own subscription, so switching means abandoning months of accumulated Azure-native expertise and project data. Azure consumption costs (OpenAI, ACA, Cosmos DB) pass through at cost to the user's own subscription; Nexus never holds or charges for cloud infrastructure, aligning incentives: we win when users build more.

ADDITIONAL INFORMATION

Full architecture documentation: 1,094-paragraph technical specification covering all 7 pipeline stages, 8 Cape Features, and 9 cross-cutting sections (security, rollback, multi-tenancy, collaboration, self-healing, supply chain trust, cost optimisation, observability, and learning mode).

Key architectural decisions and rationale:

- **Two-Layer Rollback over single-layer:** Every task writes atomically to both Cosmos DB (checkpoint) and Git (commit). Cosmos DB provides fast, granular, per-task restore from the Build Timeline UI; Git provides a full immutable history and enables standard developer workflows on the generated code. A single-layer approach would sacrifice one or the other.
- **Three-role RBAC over two-role:** Owner / Edit / View rather than a simpler Owner / Viewer split. The Edit role enables collaborative team builds where multiple contributors can make changes, but cannot transfer ownership or delete the project, which is the critical safeguard for funded team environments.
- **Field-level soft locks over document-level:** In multi-user sessions, locking only the specific Task Ledger field being edited (rather than the entire document) allows concurrent contributors to work on different sections simultaneously without blocking each other. A document-level lock in an 8-agent parallel build would create a near-constant blocking condition.
- **Ghost Handshake over strict dependency gating:** Publishing provisional schema-conformant stubs rather than blocking downstream agents on real output delivery eliminates the most common source of idle time in agent graphs, a frontend agent waiting for the backend to finalise its API shape. The cost is a targeted replay on GHOST_RESOLVED; the benefit is up to 60% reduction in total build time.
- **Token-Thrifty routing over fixed model assignment:** Assigning the cheapest model capable of completing each task with automatic escalation on failure prevents the two failure modes seen in fixed-model systems: cost explosion (all tasks on GPT-4o) and quality collapse (all tasks on Phi-4). The escalation history feeds back into the routing heuristic for future builds.
- **Impact-Based Filtering and Batch Approval:** In Manual or Semi-Automated permission modes, a naive implementation surfaces dozens of agent task notifications within minutes, causing approval fatigue where users approve without reading. Classifying tasks as Routine (silent), Notable (batched, max 10 items, 2-minute interval), or Critical (immediate interrupt) keeps the user genuinely in control without overwhelming them.

All generated code and data live in the user's own Azure subscription. Nexus operates in Zero Data Retention mode, which means no persistent access after session completion. The service has no visibility into what gets built.

Sources & References

1. Global Entrepreneurship Monitor (GEM):https://www.gemconsortium.org/economy-profiles/india
2. Press Information Bureau (PIB), Government of India: https://www.pib.gov.in/PressReleasePage.aspx?PRID=1999713
3. Press Information Bureau (PIB), Government of India: https://www.pib.gov.in/PressReleaseIframePage.aspx?PRID=2037579
4. SlashData:https://www.slashdata.co/post/global-developer-population-trends-2025-how-many-developers-are-there